Software Testing

1. Unit Testing

Unit testing checks the smallest testable part of the software, such as a single function, method or class, on its own and apart from the rest of the system. Developers write and run it while they code. It is quick to run and it finds mistakes early, when they are cheapest to fix.

2. Integration Testing

Integration testing checks that separate units or modules work correctly when they are combined. It focuses on how the parts talk to each other and on the data passed between them. It is done after unit testing.

3. System Testing

System testing checks the complete, fully integrated system against the requirements it was built to meet. It is done after integration testing is finished. System testing includes performance testing and security testing.

4. Performance Testing

Performance testing measures how fast and how stable the system is when people use it, and whether it stays responsive when it is busy.

5. Security Testing

Security testing looks for weaknesses that someone could misuse to get into the system or to reach data they should not see.

6. Smoke Testing versus Sanity Testing

Smoke testing is a quick, shallow set of tests run on a new version to confirm that the most important functions work, so the team knows it is worth testing further. Sanity testing is a narrow, focused check run after a small change or a bug fix, to confirm that the specific change works. In short, smoke testing is wide and shallow, while sanity testing is narrow and deeper.

7. Black-Box Testing versus White-Box Testing

Black-box testing checks the behaviour of the software from the outside, using inputs and expected outputs, without knowing the internal code. Two common black-box techniques are equivalence partitioning and boundary value analysis. White-box testing checks the internal structure of the code, so the tester needs to know the code, and it aims for good code coverage. In short, black-box asks whether the software does what it should, and white-box asks whether the code paths have been exercised.

8. Regression Testing

Regression testing means re-running earlier tests after a change, such as a bug fix or a new feature, to confirm that existing functionality has not been broken. It protects the software against new defects introduced by changes.